Linguaggi

Università di Verona
Imbriani Paolo -VR500437
Prof.ssa Isabella Mastroeni

March 11, 2026

Contents

1 Introduzione **3**

2 Linguaggi di programmazione **3**

2.1 Linguaggi formali 4

2.2 Aspetti di progettazione 4

2.3 Tipi di linguaggi di programmazione 5

2.3.1 Paradigmi di programmazione 5

3 Implementazione di un linguaggio di programmazione **6**

3.1 Macchina astratta per il linguaggio di programmazione 6

3.2 Interprete 7

3.2.1 Struttura dell'interprete 7

3.3 Compilatore 8

3.3.1 Struttura del compilatore 9

3.4 Specializzazione 10

4 Descrivere i linguaggi di programmazione **11**

4.1 La sintassi 11

4.1.1 Strumenti di def. della sintassi 12

1 Introduzione

Un linguaggio interpretato è un linguaggio è un linguaggio di programmazione in cui le istruzioni vengono eseguite direttamente, senza la necessità di una fase di compilazione. Un interprete è rappresentato da un modello di esecuzione cioè un CFG che rappresenta il flusso di controllo del programma. L'obiettivo è di studiare i linguaggi di programmazione e capire cosa li accomuna (insieme alle loro macro-differenze) e quale è il loro ruolo all'interno dello sviluppo. L'idea ora è capire quali cambiamenti sono stati necessari introdurre per arrivare a linguaggi di programmazione più evoluti, e capire quali sono i problemi che si sono incontrati nel corso del tempo e come sono stati risolti.

2 Linguaggi di programmazione

Da una parte abbiamo:

- **Dati**: che può essere qualsiasi cosa, numeri, stringhe qualcosa da manipolare
- **Algoritmi**: ovvero il calcolo da eseguire sui dati

Definizione 2.1: Linguaggio di programmazione

Un **linguaggio di programmazione** è uno strumento formale per descrivere algoritmi applicati ai dati.

Definizione 2.2: Programma

Un **programma** è una rappresentazione sintattica (che rispetta regole sintattiche del linguaggio di programmazione) finita di comportamenti potenzialmente infiniti. Questi comportamenti vengono definiti attraverso delle funzioni:

$$f : Dom \to Cod \cup \{\uparrow\}$$

(la funzione calcolata dall'algoritmo rappresentata/implementata attraverso il programma).

Descriviamo prima però cosa è un algoritmo:

Definizione 2.3: Algoritmo

Sequenza di passi primitivi finita descritta attraverso un programma in un linguaggio di programmazione.

2.1 Linguaggi formali

Esistono diversi tipi di linguaggi formali:

- **Linguaggio matematico**: notazione rigorosa per rappresentare funzioni:

  $$f(x) = x + 2$$

  Da questo linguaggio è nato un paradigma quello **funzionale** (λ-calcolo, ML, Haskell, Lisp...)

- **Linguaggio logico**: notazione rigorosa, regole e assiomi, calcolo come dimostrazione. Questo linguaggio però manca la descrizione di calcoli potenzialmente infiniti. Anche da qua nasce un paradigma, quello **logico** (Prolog).

- **Linguaggio di programmazione**: notazione rigorosa (grammatica), regole (semantica delle istruzioni), istruzioni che permettono di descrivere i passi di calcolo.

Dalle esigenze di diversi tipi di applicazioni, sono nati diversi tipi di linguaggi di programmazione:

- Applicazioni scientifiche come **FORTRAN**
- Applicazioni economiche come **COBOL**
- Applicazioni AI come **LISP**
- Applicazioni di sistema come **C**
- Applicazioni web come **Javascript, Java, ...**

2.2 Aspetti di progettazione

Uno degli aspetti più importanti nella progettazione di un linguaggio di programmazione sono:

- **Readability**:
  - Sintassi chiara
  - No ambiguità
  - Facilità di lettura che rende facile comprendere i programmi
  - Facilità di analisi del codice
- **Writeability**:
  - Facilità di scrittura, quanto è facile/agevole
  - Scrivere programmi in quel linguaggio, facilità di analisi del codice
- **Reliability**:

  - Conformità alle specifiche
  - Riduzione di inserimento di errori di programmazione

- **Cost**:
  - Costo di efficienza
  - Costo di apprendimento
  - Costi di utilizzo in modo corretto
  - ...

2.3 Tipi di linguaggi di programmazione

Ci sono diversi tipi di linguaggi di programmazione:

- **Linguaggi di basso livello**: vicini al linguaggio macchina come:
  - Linguaggio macchina
  - Linguaggio assembly
- **Linguaggi di alto livello**: più vicini al linguaggio umano come:
  - Linguaggi funzionali
  - Linguaggi logici
  - Linguaggi imperativi (l'unione con i linguaggi funzionale si chiama "funzionali impuri") dove vengono definite variabili come astrazione di una cella di memoria

In questi linguaggi le variabili:

- Nei linguaggi **imperativi** è un'astrazione di una cella di memoria, che può essere modificata
- Nei linguaggi **funzionali** e **matematica** è una variabile matematica e quindi **immutabile**

2.3.1 Paradigmi di programmazione

Nei linguaggi ci sono i seguenti paradigmi:

- **Object-oriented**: dove i programmi sono organizzati in classi che rappresentano oggetti, con attributi e metodi.
- **Procedurali**: dove i programmi sono organizzati in procedure o funzioni che eseguono compiti specifici.
- **Strutturata**: dove i programmi sono organizzati in blocchi di codice
- **Dichiarativi**: si basano sulle basi matematiche e logiche, dove i programmi sono organizzati in dichiarazioni che descrivono ciò che deve essere fatto, piuttosto che come farlo.

3 Implementazione di un linguaggio di programmazione

Per implementare un linguaggio di programmazione, è necessario:

- **Architettura** della macchina su cui è implementato
- **Metodologie di progettazione**: cambiare da un linguaggio ad oggetti a un linguaggio dichiarativo determina cambiamenti nella progettazione del linguaggio e quindi anche nella sua implementazione

3.1 Macchina astratta per il linguaggio di programmazione

Abbiamo bisogno di una macchina astratta per il linguaggio di programmazione, che è un modello di esecuzione che rappresenta il flusso di controllo del programma.

Definizione 3.1: Macchina astratta

Dato un LP L, una M_L (macchina astratta per L) è un insieme di **strutture dati** e **algoritmi** che permettono di memorizzare e eseguire programmi scritti in L. Intuitivamente, M_L è la macchina il cui linguaggio è esattamente L.

Definizione 3.2: Linguaggio macchina

Dato una macchina M il linguaggio L_m (linguaggio macchina di M) è il linguaggio che ha come stringhe quelle direttamente interpretabili da M.
Si ha una pila di macchine astratte M_{i-1}, M_i, M_{i+1} e i linguaggi macchina $L_{m_{i-1}}, L_{m_i}, L_{m_{i+1}}$. Il linguaggio L_{m_i} può essere usato ai livelli superiori.

Teorema 3.1.1

Per ogni M_L esiste un unico linguaggio che è il linguaggio riconosciuto ed eseguito da M_L (linguaggio macchina di M_L). Per ogni LP possono esister infinite macchine astratte per L.

Se immaginiamo le macchine astratte a più livelli, le macchine astratte di livello più basso donano i loro servizi ai livelli superiori che usano i linguaggi ad alto livello per implementare funzioni più complesse. Questo compongono diversi **livelli di astrazione**.

$$\text{Implementare } L \equiv \text{Realizzare } M_L$$

Per realizzare M_L si può:

- Realizzare M_L vuol dire tradurre L in un linguaggio macchina sottostante. (Compilazione)

- Oppure si può eseguire L sulla macchina sottostante. (Interpretazione)

Definizione 3.3: Compilazione

Dati due linguaggi L, L' una **compilazione** vuol dire fare una traduzione esplicita tra L e L' o viceversa.

Definizione 3.4: Interpretazione

Dati due linguaggi L, L' una **interpretazione** vuol dire fare una traduzione implicita tra L in L' ovvero simuliamo l'esecuzione di L attraverso L'.

3.2 Interprete

Per definire cosa è un interprete dobbiamo introdurre le seguenti notazioni:

- $Prog^L$: insieme dei programmi scritti in L
- D: insieme dei dati manipolati dai programmi in L
- P^L un singolo programma in L. (Quindi $P^L \in Prog^L$)

Definizione 3.5

Dato L un linguaggio di programmazione, un **interprete** per L (scritto in L_0) è un programma

$$I^{LoL} : (Prog^L \times D) \to D$$

L'inteprete è la macchina di Turing universale.

$$\underbrace{I^{LoL}(P^L, input) = output \text{ t.c } P^L(input) = output}_{I^{LoL}(P^L, input) = P^L(input)}$$

3.2.1 Struttura dell'interprete

Vediamo ora come un generico interprete è strutturato, e quali sono le sue componenti principali. Lo scriveremo in pseudocodice, e vedremo che è composto da tre fasi principali: Fetch, Decode, Execute.

```
go := true;
while go do
    fetch (opcode, opinfo) from PC;
    decode (opcode, opinfo);
    if opccode needs arguments
        then fetch arguments from PC;
```

```
case opcode of
    opcode1: execute opcode1;
    opcode2: execute opcode2;
    ...
    opcodeN: execute opcodeN;
    halt: go := false;

if opcode has res then store(res)
pc = pc + sizeof(opcode) + sizeof(arguments);
```

Esempio 3.1

Un **autointerprete** è un interprete scritto nello stesso linguaggio che interpreta. Vediamo un esempio di codice:

```
input P, d; // Programma che deve essere interpretato e il suo dato
pc := 2; store := [in -> d, out -> 0, x_1 -> 0,...]
while pc < length(P) do {
    instruction := lookup(P, pc); // Trova l'istruzione da eseguire
    case instruction of // Dispatch on syntax
    skip : pc++;
    x := e : store := store [x -> eval(e, store)]; pc++;
    ... endw;
}
output(e, store) = case e of
    costant: e
    variable: store(e)
    e1 + e2: eval(e1, store) + eval(e2, store)
    e1 - e2: eval(e1, store) - eval(e2, store)
    e1 * e2: eval(e1, store) * eval(e2, store)
    ...
end
```

3.3 Compilatore

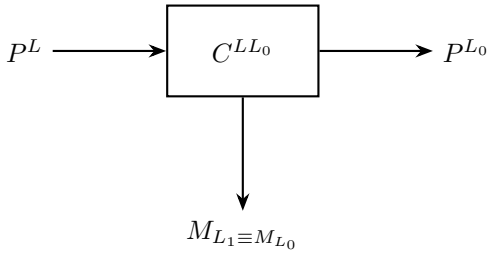
Definizione 3.6: Compilatore

Un compilatore da L a L_0 è un programma

$$C^{LL_0} : Prog^L \to Prog^{L_0}$$

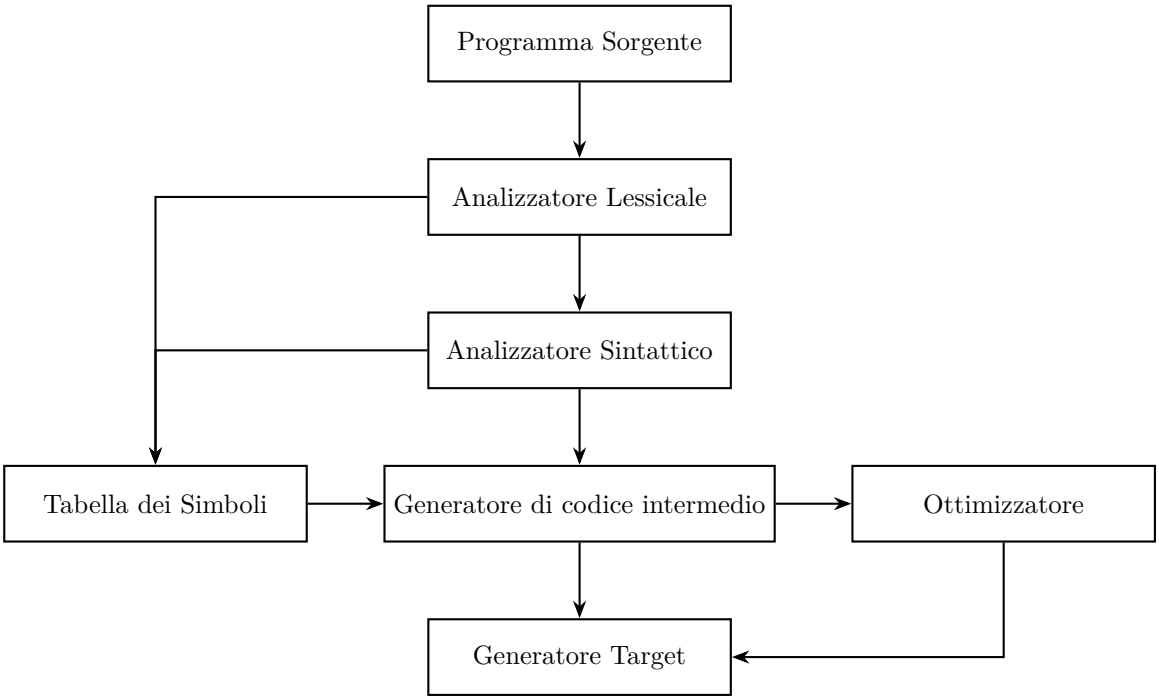
Dove

$$\forall P^L \in Prog^L \,.\, C^{LL_0}(P^L) = P^{L_0} \text{ t.c. } \forall d \in D \,.\, P^L(d) = P^{L_0}(d)$$



Traduce in un codice diverso e separa la traduzione dall'esecuzione.

3.3.1 Struttura del compilatore



Dove rispettivamente:

- Il programma sorgente è il programma scritto in L che deve essere compilato
- L'analizzatore lessicale è la fase che prende in input il programma sorgente e lo suddivide in token, ovvero unità lessicali come parole chiave, identificatori, operatori, ecc.
- L'analizzatore sintattico prende i token prodotti dall'analizzatore lessicale e costruisce una rappresentazione ad albero del programma, chiamata albero sintattico o AST (Abstract Syntax Tree).

- Il generatore di codice prende l'albero sintattico e produce un codice intermedio o direttamente il codice target.
- La tabella dei simboli è una struttura dati che memorizza informazioni sui simboli utilizzati nel programma, come variabili, funzioni, classi, ecc. Viene utilizzata durante l'analisi semantica e la generazione del codice.
- l'ottimizzatore migliora il codice generato in modo da renderlo più efficiente, ad esempio eliminando codice morto, riducendo il numero di istruzioni, migliorando l'uso della memoria, ecc.
- il generatore target prende il codice intermedio e lo traduce in codice eseguibile per la macchina target,

Ci sono alcune modifiche che agevolano l'ultima parte di generazione del codice e che ottimizza il codice generato in modo da renderlo più efficiente.

Esempio 3.2

Considerando M_L con L PL da implementare e M_0 con L_0 PL eseguibile.
L'interprete:

$$M_L \Longleftarrow M_i \Longleftarrow M_0$$

Il compilatore:

$$M_L \Longrightarrow M_i \Longrightarrow M_0$$

3.4 Specializzazione

Definizione 3.7: Specializzatore

Uno specializzatore per un PL L è un programma che realizza la funzione:

$$Spec_L : Prog^L \times D \to Prog^L$$

tale che

$$\forall P^L \in Prog^L, \forall d \in D$$

$$Spec_L(P^L, d) = P^L_d \text{ t.c. } P^L(d, y) = P^L_d(y)$$

Osservazione 3.1

Lo specializzatore fornisce un legame tra interprete e compilatore. Specializzando un interprete (scritto in L_0) sul programma (Scritto in L) otteniamo un **compilatore** da L

a L_0. Dove I è scritto in L_0:

$Spec(I, P^L)$ è un programma scritto in L_0 che integra nel codice l'esecuzione di PL

4 Descrivere i linguaggi di programmazione

I linguaggi di programmazione sono pur sempre qualcosa di simili a quelli naturali con la quale condividono alcuni elementi in comune come:

- **La sintassi**: l'insieme delle regole di formazione (determina quali sono le frasi ben formate nel linguaggio) descrive relazioni tra simboli;
- **La semantica**: Attribuzione del significato ai simboli (determina il significato delle frasi ben formate nel linguaggio) descrive relazioni tra simboli e significati;
- **La pragmatica**: regole di buon utilizzo del linguaggio;
- **L'implementazione**: esistono strumenti che eseguento frasi ben formate nel linguaggio, e che quindi ne permettono l'utilizzo (interpreti, compilatori).

4.1 La sintassi

Definizione 4.1: Parola

La parola è definita come una stringa in un alfabeto finito Σ ed è un oggetto atomico.

Definizione 4.2: Frase

La frase è definita come una sequenza di parole (ben formata ovvero che segue le regole della sintassi).

Definizione 4.3: Linguaggio

Il **linguaggio** è un insieme di frasi ben formate.

Esempio 4.1

Per esempio, in fondamenti avevamo i visto i seguenti:

$$\text{Il linguaggio} = \{a^n b^n \mid n \in \mathbb{N}\} \text{ con } \Sigma = \{a, b\}$$

Dove le stringhe possono essere $\varepsilon, ab, a^2b^2, \ldots$ Per i linguaggi di programmazione

$$\Sigma = \text{if, while, then, else}$$

Le stringhe sono i programmi (sequenza di parole) e il linguaggio è il nostro linguaggio di programmazione.

Definizione 4.4: Lessema

Un **lessema** è una parola che ha significato (costituito da soli terminali nell'alfabeto)

Esempio 4.2

Nella frase "if exp then com else com":

- if, then, else, sono lessemi
- exp, com non sono lessemi (non hanno significato)

Definizione 4.5: Token

I **token** sono categorie sintattiche (simboli non terminali). Nell'esempio (4.2)

- if, then, else sono lessemi
- exp, com sono token (categorie sintattiche)

Esempio 4.3

Nella frase "if x > 0 then $x = x + 1$ else $x = x - 1$":

- if, then, else sono lessemi
- $x > 0$, $x = x + 1$, $x = x - 1$ sono token (categorie sintattiche)

4.1.1 Strumenti di def. della sintassi

Definizione 4.6: Riconoscitore

Il **riconoscitore** è uno strumento per il riconoscimento di stringhe appartenti al linguaggio.

In Fondamenti, abbiamo visto come il riconoscitore più semplice per un linguaggio è rappresentato da un automa a stati finiti e li ritroviamo nell'analisi lessicale per riconoscimento dei lessemi.

Definizione 4.7: Generatore

Il **generatore** è uno strumento per la generazione di stringhe appartenenti al linguaggio.

Il generatore più semplice per un linguaggio è rappresentato da una grammatica, utilizzate nell'analisi sintattica dell'albero di parsing. Una grammatica CF per i PL è $G = (V, \Sigma, R, S)$ dove:

- V è un insieme finito di categorie sintattiche
- Σ è un insieme di vocabolario parole ($V \cap T = \varnothing$)
- R è un insieme di regole di regole di produzione

$$A \to \alpha,\ A \in V,\ \alpha \in (V \cup T)^*$$

- $S \in V$ è la categoria della frase (programma)

Si utilizza la notazione BNF (CF) spesso usato per i PL:

- $\rightsquigarrow \equiv ::=$

Esempio 4.4

Supponiamo di avere una grammatica per le espressioni booleane:

$$G =< \{Exp\} >, \{1, 0, not, and, or, (,)\}, P, Exp >$$

Dove Exp può essere:

- $Exp \to 0$, $Exp \to 1$,
- $Exp \to not\ Exp$, $not\ Exp$, $Exp\ and\ Exp$, $Exp\ or\ Exp$

quindi:

$$Exp \to 0 \mid 1 \mid (Exp\ or\ Exp) \mid (Exp\ and\ Exp) \mid (not\ Exp)$$